DIGITAL DESIGN · PROJECT 1

# Spartan-6 DSP48A1 Slice Design




Verilog RTL · QuestaSim · Vivado · Questa Lint

SUBMITTED BY

## Ahmed Mohamed Khalafallah

| | |
|---|---|
| **Target device** | xc7a200tffg1156-3 (Artix-7) |
| **Clock constraint** | 100 MHz · pin W5 |
| **Simulator** | QuestaSim |
| **Synthesis / P&R** | Vivado |
| **Linting** | Questa Lint (default methodology & goals) |

# Contents

**Results at a Glance** . . . . . 3
**Introduction** . . . . . 4
**1. RTL Code** . . . . . 8
**2. Testbench Code** . . . . . 15
**3. Do File** . . . . . 23
**4. QuestaSim Snippets** . . . . . 25
**5. Constraint File** . . . . . 26
**6. Elaboration** . . . . . 27
**7. Synthesis** . . . . . 29
**8. Implementation** . . . . . 33
**9. Linting** . . . . . 36

**How to read this report.** The Introduction explains what the DSP48A1 slice is and how the testbench scenarios are defined. Sections 1–9 then follow the required deliverable order: RTL, testbench, do file, QuestaSim waveforms, constraints, elaboration, synthesis, implementation and linting.

# Results at a Glance

Everything below is backed by the screenshots in sections 4 to 9

| Stage | Result | Where |
|---|---|---|
| **Simulation** (QuestaSim) | Self-checking testbench: **28 checks passed, 0 failed** across reset + 4 DSP paths | Sections 3–4 |
| **Elaboration** | 17 cells, 346 I/O ports. 23 warnings, all unused-port warnings from default parameters | Section 6 |
| **Synthesis** | 255 LUT, 160 FF, **1 DSP48E1**, 327 IO. WNS +5.216 ns, WHS +0.182 ns, all constraints met | Section 7 |
| **Implementation** | 254 LUT, 179 FF, **1 DSP48E1**, 327 IO. WNS +3.696 ns, WHS +0.271 ns, all constraints met | Section 8 |
| **Linting** (Questa Lint) | 0 errors, 0 warnings. Only 5 info/style items; none remain after waiving | Section 9 |

**Timing at 100 MHz.** The 10 ns clock leaves a comfortable positive setup slack after implementation (+3.696 ns), so the design could run noticeably faster than 100 MHz on this device.

# Introduction

What is the DSP48A1 slice?

The DSP48A1 is the hard arithmetic block of Xilinx (now AMD) **Spartan-6** FPGAs. One slice contains an 18-bit pre-adder/subtracter, an 18×18 signed multiplier and a 48-bit post-adder/subtracter, wrapped in optional pipeline registers. This makes it ideal for filters, multiply-accumulate (MAC) loops and wide additions. The project implements this slice in Verilog, with parameters that decide how many pipeline registers are used and whether resets are synchronous or asynchronous.

## Block diagram (ports)

The slice has 18-bit A, B and D inputs, a 48-bit C input, an 8-bit OPMODE control, individual resets and clock enables for every register group, and the outputs P, M, BCOUT, PCOUT, CARRYOUT and CARRYOUTF.

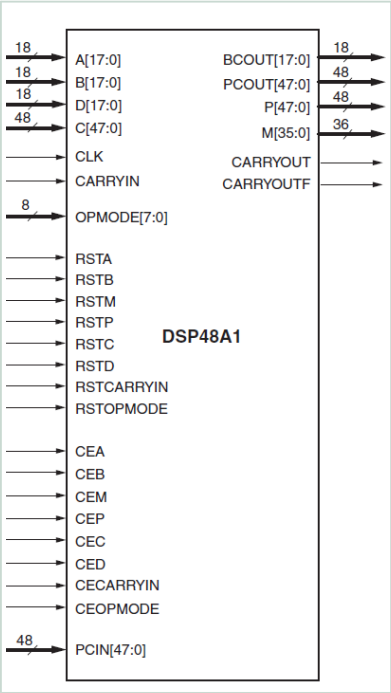



*Figure I.1: DSP48A1 external ports (source: project specification).*

## Port groups

| Group | Signals | Role |
|---|---|---|
| **Data in** | `A, B, D [17:0], C [47:0], CARRYIN, BCIN, PCIN` | Operands and cascade inputs |
| **Control** | `CLK, OPMODE[7:0]` | Clock and arithmetic mode select |
| **Clock enables** | `CEA, CEB, CEC, CED, CEM, CEP, CECARRYIN, CEOPMODE` | One enable per register group |
| **Resets** | `RSTA, RSTB, RSTC, RSTD, RSTM, RSTP, RSTCARRYIN, RSTOPMODE` | Active-high, sync or async per RSTTYPE |
| **Outputs** | `P, PCOUT [47:0], M [35:0], BCOUT [17:0], CARRYOUT, CARRYOUTF` | Results and cascade outputs |

## Internal data path

Data flows from left to right. Each register has a bypass multiplexer (grey), fixed at configuration time by a parameter. The X, Z, pre-adder and post-adder behaviours are selected at run time by OPMODE.

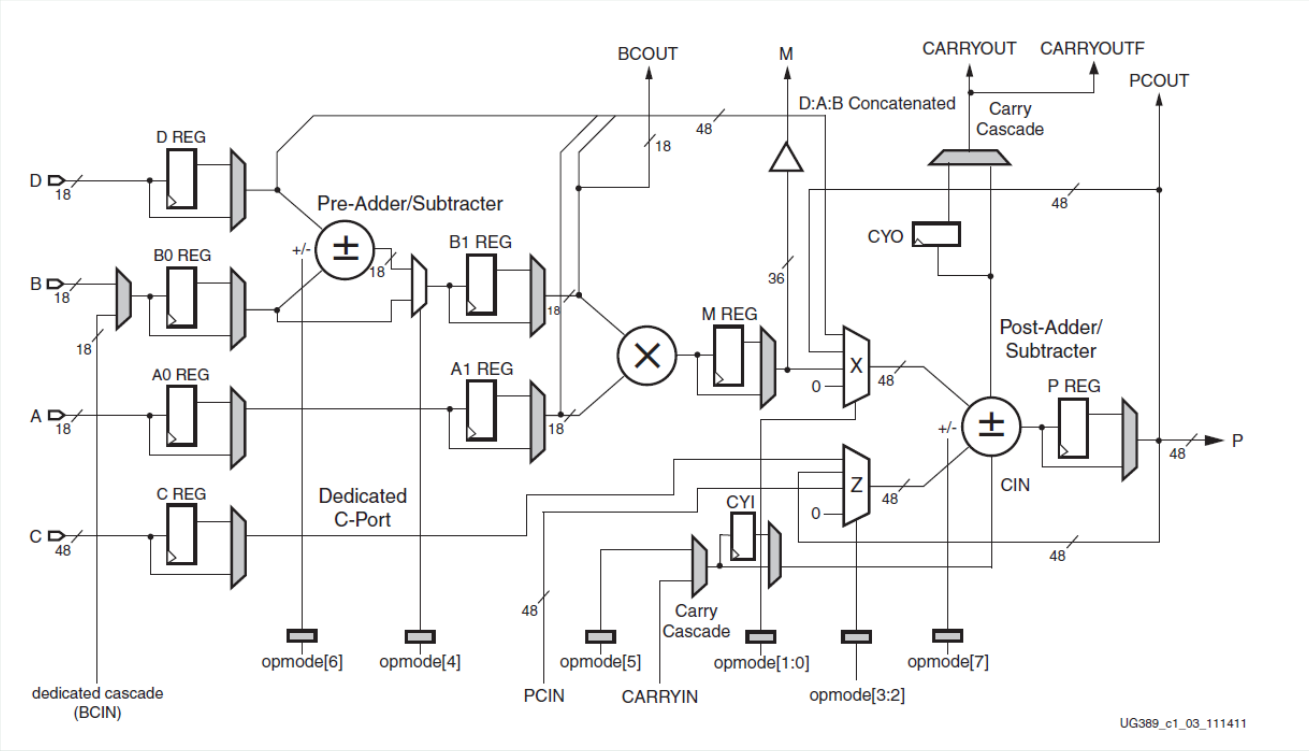



*Figure I.2: DSP48A1 internal architecture (Xilinx UG389 Figure 1-3).*

### Reading the diagram

- **Input stage:** D, B0/B1, A0/A1, C and OPMODE each have a register. A0/B0 are first-stage and A1/B1 are second-stage registers.
- **Pre-adder:** computes D±B (OPMODE[6] chooses add or subtract). OPMODE[4] decides whether the multiplier sees this result or plain B.
- **Multiplier:** 18×18 signed, producing the 36-bit value M, optionally registered in MREG.
- **X and Z muxes (48-bit):** X is chosen by OPMODE[1:0] (0, M, P, D:A:B). Z is chosen by OPMODE[3:2] (0, PCIN, P, C).
- **Post-adder:** computes Z ± (X + CIN). OPMODE[7] selects subtraction. CIN comes from CARRYIN or OPMODE[5], depending on CARRYINSEL.
- **Outputs:** P (and PCOUT) from the P register, CARRYOUT/CARRYOUTF from the CYO register, BCOUT from B1, M from the multiplier stage.

## OPMODE encoding

| Port Name | Function |
|---|---|
| OPMODE[1:0] | **Specifies the source of the X input to the post-adder/subtracter** |
| | 0 – Specifies to place all zeros (disable the post-adder/subtracter and propagate the Z result to P)<br>1 – Use the multiplier product<br>2 – Use the P output signal (accumulator)<br>3 – Use the concatenated D:A:B input signals |
| OPMODE[3:2] | **Specifies the source of the Z input to the post-adder/subtracter** |
| | 0 – Specifies to place all zeros (disable the post-adder/subtracter and propagate the multiplier product or other X result to P)<br>1 – Use the PCIN<br>2 – Use the P output signal (accumulator)<br>3 – Use the C port |
| OPMODE[4] | **Specifies the use of the pre-adder/subtracter** |
| | 0 – Bypass the pre-adder supplying the data on port B directly to the multiplier<br>1 – Selects to use the pre-adder adding or subtracting the values on the B and D ports prior to the multiplier |
| OPMODE[5] | **Forces a value on the carry input of the carry-in register (CYI) or direct to the CIN to the post-adder. Only applicable when CARRYINSEL = OPMODE5** |
| OPMODE[6] | **Specifies whether the pre-adder/subtracter is an adder or subtracter** |
| | 0 – Specifies pre-adder/subtracter to perform an addition operation<br>1 – Specifies pre-adder/subtracter to perform a subtraction operation (D-B) |
| OPMODE[7] | **Specifies whether the post-adder/subtracter is an adder or subtracter** |
| | 0 – Specifies post-adder/subtracter to perform an addition operation<br>1 – Specifies post-adder/subtracter to perform a subtraction operation (Z-(X+CIN)) |

*Figure I.3: OPMODE bit functions (source: project specification).*

## Parameters and default values used in this project

| Parameter | Default | Meaning |
|---|---|---|
| `A0REG, B0REG` | `0` | First-stage A/B registers: 0 = bypass (no register) |
| `A1REG, B1REG` | `1` | Second-stage A/B registers: 1 = registered |
| `CREG, DREG, MREG, PREG` | `1` | Registers on C, D, multiplier output and P output |
| `CARRYINREG, CARRYOUTREG` | `1` | Registers on the carry-in (CYI) and carry-out (CYO) paths |
| `OPMODEREG` | `1` | Register on the OPMODE control word |
| `CARRYINSEL` | `"OPMODE5"` | Carry-in source: "CARRYIN" input or OPMODE[5]; otherwise tied to 0 |
| `B_INPUT` | `"DIRECT"` | B port source: "DIRECT" (B) or "CASCADE" (BCIN); otherwise tied to 0 |
| `RSTTYPE` | `"SYNC"` | All resets synchronous ("SYNC") or asynchronous ("ASYNC"), active high |

## How this implementation is organised

Instead of repeating generate blocks for every register, the design is built from four small reusable modules and one top level. The same `pipe_reg` module is instantiated for every register stage.

| File / module | Purpose |
|---|---|
| `pipe_reg.v` | Generic register with parameters WIDTH, REG_EN (1 = register, 0 = combinational bypass) and RSTTYPE (SYNC/ASYNC), plus clock enable and reset |
| `mux4.v` | Parameterised 4-to-1 multiplexer, used for the X mux (OPMODE[1:0]) and the Z mux (OPMODE[3:2]) |
| `preadder_sub.v` | Computes D + B or D − B, selected by OPMODE[6] |
| `postadder_sub.v` | Computes Z + X + CIN or Z − (X + CIN), selected by OPMODE[7], and produces the carry-out bit |
| `dsp48a1_top.v` | Top module DSP48A1: wires input registers, pre-adder, multiplier, X/Z muxes, post-adder and output registers together |
| `dsp48a1_tb_.v` | Self-checking testbench with reset check and four directed DSP paths |
| `run_.do` | QuestaSim script: compile, simulate, add waves, run |
| `constraints_1.xdc` | Clock constraint: 100 MHz on pin W5 |

# 1 RTL Code

Verilog sources: four helper modules and the DSP48A1 top level

## 1.1 mux4.v

A 4-to-1 multiplexer with a parameterised width (48 bits by default). A single case statement selects among four inputs; it is reused for both the X and Z multiplexers.

```
1  // mux4 : Generic 4-to-1 multiplexer, parameterizable width.
2  // Used for both the X mux (opmode[1:0]) and the Z mux (opmode[3:2]).
3  `timescale 1ns/1ps
4
5  module mux4 #(
6      parameter WIDTH = 48
7  )(
8      input  wire [1:0]       sel,
9      input  wire [WIDTH-1:0] in0,
10     input  wire [WIDTH-1:0] in1,
11     input  wire [WIDTH-1:0] in2,
12     input  wire [WIDTH-1:0] in3,
13     output reg  [WIDTH-1:0] out
14 );
15
16     always @(*) begin
17         case (sel)
18             2'b00: out = in0;
19             2'b01: out = in1;
20             2'b10: out = in2;
21             default: out = in3;
22         endcase
23     end
24
25 endmodule
26
```

*Listing 1.1: mux4.v*

## 1.2 pipe_reg.v

The building block for every register stage. With `REG_EN = 1` a flip-flop is generated, with synchronous or asynchronous reset chosen by RSTTYPE and a clock enable. With `REG_EN = 0` the module becomes a plain wire, which is how A0REG/B0REG bypass works.

```
// pipe_reg : Generic pipeline register with optional bypass,
//            optional register, and sync/async reset.
// Used for EVERY register stage in the DSP48A1 (D, A0, A1, B0, B1,
// C, OPMODE, M, CARRYIN, P, CARRYOUT)
`timescale 1ns/1ps

module pipe_reg #(
    parameter WIDTH   = 18,
    parameter REG_EN  = 1,       // 1 = real register, 0 = combinational bypass
    parameter RSTTYPE = "SYNC"   // "SYNC" or "ASYNC"
)(
    input  wire             CLK,
    input  wire             RST,
    input  wire             CE,
    input  wire [WIDTH-1:0] D,
    output reg  [WIDTH-1:0] Q
);

    generate
        if (REG_EN) begin : g_registered
            if (RSTTYPE == "ASYNC") begin : g_async
                always @(posedge CLK or posedge RST)
                    if (RST) Q <= {WIDTH{1'b0}};
                    else if (CE) Q <= D;
            end else begin : g_sync
                always @(posedge CLK)
                    if (RST) Q <= {WIDTH{1'b0}};
                    else if (CE) Q <= D;
            end
        end else begin : g_bypass
            always @(*) Q = D;
        end
    endgenerate

endmodule
```

*Listing 1.2: pipe_reg.v*

## 1.3 postadder_sub.v and preadder_sub.v

Both arithmetic blocks are combinational. The post-adder widens its operands to 49 bits so that the carry/borrow bit is available, then picks the add or subtract result with OPMODE[7]. The pre-adder is the 18-bit D±B stage.

```
// postadder_sub : Z +/- (X + CIN)  (opmode[7] selects add=0 / sub=1)
// Computes both add and sub results, then picks one, along with carry.
`timescale 1ns/1ps

module postadder_sub (
    input  wire [47:0] Z,
    input  wire [47:0] X,
    input  wire        CIN,
    input  wire        sub,      // opmode[7]
    output wire [47:0] result,
    output wire        carry
);

    wire [48:0] add_ext = {1'b0, Z} + {1'b0, X} + CIN;
    wire [48:0] sub_ext = {1'b0, Z} - ({1'b0, X} + CIN);

    assign result = sub ? sub_ext[47:0] : add_ext[47:0];
    assign carry  = sub ? sub_ext[48]   : add_ext[48];

endmodule

```

*Listing 1.3: postadder_sub.v*

```
// preadder_sub : D +/- B  (opmode[6] selects add=0 / sub=1)
`timescale 1ns/1ps

module preadder_sub (
    input  wire [17:0] D,
    input  wire [17:0] B,
    input  wire        sub,     // opmode[6]
    output wire [17:0] result
);

    assign result = sub ? (D - B) : (D + B);

endmodule

```

*Listing 1.4: preadder_sub.v*

## 1.4 dsp48a1_top.v (top-level module)

The top level declares the parameters and ports, then connects the submodules. It is shown in four consecutive screenshots (parts 1–4); the first and last lines of neighbouring parts overlap slightly where the editor view was scrolled.

### Part 1: parameters and port list

```
1  // DSP48A1 built from submodules (pipe_reg, mux4,
2  // preadder_sub, postadder_sub) instead of repeating generate blocks.
3  `timescale 1ns/1ps
4
5  module DSP48A1 #(
6      parameter A0REG        = 0,
7      parameter A1REG        = 1,
8      parameter B0REG        = 0,
9      parameter B1REG        = 1,
10     parameter CREG         = 1,
11     parameter DREG         = 1,
12     parameter MREG         = 1,
13     parameter PREG         = 1,
14     parameter CARRYINREG   = 1,
15     parameter CARRYOUTREG  = 1,
16     parameter OPMODEREG    = 1,
17     parameter CARRYINSEL   = "OPMODE5",
18     parameter B_INPUT      = "DIRECT",
19     parameter RSTTYPE      = "SYNC"
20 )(
21     input  wire [17:0] A,
22     input  wire [17:0] B,
23     input  wire [17:0] D,
24     input  wire [47:0] C,
25     input  wire        CLK,
26     input  wire        CARRYIN,
27     input  wire [7:0]  OPMODE,
28     input  wire [17:0] BCIN,
29     input  wire [47:0] PCIN,
30
31     input  wire RSTA, RSTB, RSTM, RSTP, RSTC, RSTD, RSTCARRYIN, RSTOPMODE,
32     input  wire CEA, CEB, CEM, CEP, CEC, CED, CECARRYIN, CEOPMODE,
33
34     output wire [17:0] BCOUT,
35     output wire [47:0] PCOUT,
36     output wire [47:0] P,
37     output wire [35:0] M,
38     output wire        CARRYOUT,
```

*Listing 1.5: dsp48a1_top.v, lines 1–38*

## Part 2: internal wires and first-stage registers

```
39      output wire         CARRYOUTF
40  );
41      // Internal wires between pipeline stages
42      wire [17:0] b_src;
43      wire [17:0] d_reg_out;
44      wire [17:0] a0_reg_out, a1_reg_out;
45      wire [17:0] b0_reg_out, b1_reg_out, b1_in;
46      wire [17:0] preadd_result;
47      wire [47:0] c_reg_out;
48      wire [7:0]  opmode_use;
49      wire [35:0] mult_result, m_out;
50      wire        cyi_in, cin_final;
51      wire        post_carry;
52      wire [47:0] x_val, z_val, post_result, p_reg_out;
53
54      // B input source (DIRECT vs CASCADE)
55      assign b_src = (B_INPUT == "CASCADE") ? BCIN : B;
56
57      // Stage 1 registers - ALL are just pipe_reg instances now
58      pipe_reg #(.WIDTH(18), .REG_EN(DREG), .RSTTYPE(RSTTYPE))
59          u_dreg (.CLK(CLK), .RST(RSTD), .CE(CED), .D(D), .Q(d_reg_out));
60
61      pipe_reg #(.WIDTH(18), .REG_EN(A0REG), .RSTTYPE(RSTTYPE))
62          u_a0reg (.CLK(CLK), .RST(RSTA), .CE(CEA), .D(A), .Q(a0_reg_out));
63
64      pipe_reg #(.WIDTH(18), .REG_EN(A1REG), .RSTTYPE(RSTTYPE))
65          u_a1reg (.CLK(CLK), .RST(RSTA), .CE(CEA), .D(a0_reg_out), .Q(a1_reg_out));
66
67      pipe_reg #(.WIDTH(18), .REG_EN(B0REG), .RSTTYPE(RSTTYPE))
68          u_b0reg (.CLK(CLK), .RST(RSTB), .CE(CEB), .D(b_src), .Q(b0_reg_out));
69
70      pipe_reg #(.WIDTH(48), .REG_EN(CREG), .RSTTYPE(RSTTYPE))
71          u_creg (.CLK(CLK), .RST(RSTC), .CE(CEC), .D(C), .Q(c_reg_out));
72
73      pipe_reg #(.WIDTH(8), .REG_EN(OPMODEREG), .RSTTYPE(RSTTYPE))
74          u_opmodereg (.CLK(CLK), .RST(RSTOPMODE), .CE(CEOPMODE), .D(OPMODE), .Q(opmode_use));
```

*Listing 1.6: dsp48a1_top.v, lines 39–74*

## Part 3: pre-adder, multiplier, carry-in and X mux

```
74          u_opmodereg (.CLK(CLK), .RST(RSTOPMODE), .CE(CEOPMODE), .D(OPMODE), .Q(opmode_use));
75
76      // Pre-adder/subtracter + bypass mux (opmode[4])
77      preadder_sub u_preadder (
78          .D(d_reg_out), .B(b0_reg_out), .sub(opmode_use[6]), .result(preadd_result)
79      );
80
81      assign b1_in = opmode_use[4] ? preadd_result : b0_reg_out;
82
83      pipe_reg #(.WIDTH(18), .REG_EN(B1REG), .RSTTYPE(RSTTYPE))
84          u_b1reg (.CLK(CLK), .RST(RSTB), .CE(CEB), .D(b1_in), .Q(b1_reg_out));
85
86      assign BCOUT = b1_reg_out;
87
88      // Multiplier
89      assign mult_result = $signed(a1_reg_out) * $signed(b1_reg_out);
90
91      pipe_reg #(.WIDTH(36), .REG_EN(MREG), .RSTTYPE(RSTTYPE))
92          u_mreg (.CLK(CLK), .RST(RSTM), .CE(CEM), .D(mult_result), .Q(m_out));
93
94      assign M = m_out;
95
96      // Carry-in select + register
97      assign cyi_in = (CARRYINSEL == "CARRYIN") ? CARRYIN :
98                      (CARRYINSEL == "OPMODE5") ? opmode_use[5] : 1'b0;
99
100     pipe_reg #(.WIDTH(1), .REG_EN(CARRYINREG), .RSTTYPE(RSTTYPE))
101         u_cyireg (.CLK(CLK), .RST(RSTCARRYIN), .CE(CECARRYIN), .D(cyi_in), .Q(cin_final));
102
103     // X mux (opmode[1:0]) and Z mux (opmode[3:2])
104     mux4 #(.WIDTH(48)) u_xmux (
105         .sel(opmode_use[1:0]),
106         .in0(48'b0),
107         .in1({{12{m_out[35]}}, m_out}),
108         .in2(p_reg_out),
109         .in3({d_reg_out[11:0], a1_reg_out, b1_reg_out}),
110         .out(x_val)
```

*Listing 1.7: dsp48a1_top.v, lines 74–110*

## Part 4: Z mux, post-adder, P register and carry-out

```
110         .out(x_val)
111     );
112
113     mux4 #(.WIDTH(48)) u_zmux (
114         .sel(opmode_use[3:2]),
115         .in0(48'b0),
116         .in1(PCIN),
117         .in2(p_reg_out),
118         .in3(c_reg_out),
119         .out(z_val)
120     );
121
122     // Post-adder/subtracter
123     postadder_sub u_postadder (
124         .Z(z_val), .X(x_val), .CIN(cin_final), .sub(opmode_use[7]),
125         .result(post_result), .carry(post_carry)
126     );
127
128     pipe_reg #(.WIDTH(48), .REG_EN(PREG), .RSTTYPE(RSTTYPE))
129         u_preg (.CLK(CLK), .RST(RSTP), .CE(CEP), .D(post_result), .Q(p_reg_out));
130
131     assign P     = p_reg_out;
132     assign PCOUT = p_reg_out;
133
134     // Carry-out register
135     pipe_reg #(.WIDTH(1), .REG_EN(CARRYOUTREG), .RSTTYPE(RSTTYPE))
136         u_cyoreg (.CLK(CLK), .RST(RSTCARRYIN), .CE(CECARRYIN), .D(post_carry), .Q(CARRYOUT));
137
138     assign CARRYOUTF = CARRYOUT;
139
140 endmodule
141
```

*Listing 1.8: dsp48a1_top.v, lines 110–141*

# 2 Testbench Code

Test plan, data-flow figures, and the self-checking testbench

## 2.0 Test plan

The testbench keeps all parameters at their defaults and applies one reset check followed by four directed DSP paths. Each path drives fixed values on A, B, C, D (and PCIN in path 4), random values on the unused cascade inputs, waits for the data to travel through the pipeline, and then compares every output with the expected value. The latency is counted in negative clock edges.

| Test | What it verifies | OPMODE | Wait | Expected outputs |
|---|---|---|---|---|
| **2.1** Reset | All resets high, other inputs random: every output must be 0. Then resets are released and all clock enables asserted | `random` | 1 edge | P = PCOUT = M = BCOUT = 0, CARRYOUT = CARRYOUTF = 0 |
| **2.2** Path 1 | Pre-subtract, multiply, post-subtract: X = M, Z = C | `8'b11011101` | 4 edges | BCOUT = 0xF, M = 0x12C, P = PCOUT = 0x32, CARRYOUT = 0 |
| **2.3** Path 2 | Pre-add and multiply; X and Z forced to 0 | `8'b00010000` | 3 edges | BCOUT = 0x23, M = 0x2BC, P = PCOUT = 0, CARRYOUT = 0 |
| **2.4** Path 3 | No pre-adder; P fed back through X and Z (accumulate) | `8'b00001010` | 3 edges | BCOUT = 0xA, M = 0xC8, P = PCOUT = previous P, CARRYOUT = previous value |
| **2.5** Path 4 | No pre-adder; D:A:B through X, PCIN through Z, post-subtract with carry-in = 1 | `8'b10100111` | 3 edges | BCOUT = 0x6, M = 0x1E, P = PCOUT = 0xFE6FFFEC0BB1, CARRYOUT = 1 |

Inputs used: A = 20, B = 10, C = 350, D = 25 for paths 1–3; A = 5, B = 6, C = 350, D = 25, PCIN = 3000 for path 4.

## 2.1 Verify reset operation

All eight active-high resets are asserted while the other inputs receive random values. After a negative clock edge, every output is checked to be zero. The resets are then released and all clock enables are asserted for the paths that follow.

## 2.2 Verify DSP path 1: pre-subtract → multiply → post-subtract

**OPMODE = 8'b11011101.** Bit 7 = 1 (post-subtract), bit 6 = 1 (D − B), bit 4 = 1 (pre-adder used), bits [3:2] = 11 (Z = C), bits [1:0] = 01 (X = M). Expected: D − B = 15 (0xF), M = 15 × 20 = 300 (0x12C), P = C − M = 350 − 300 = 50 (0x32). The red arrows show the active path through four register stages.

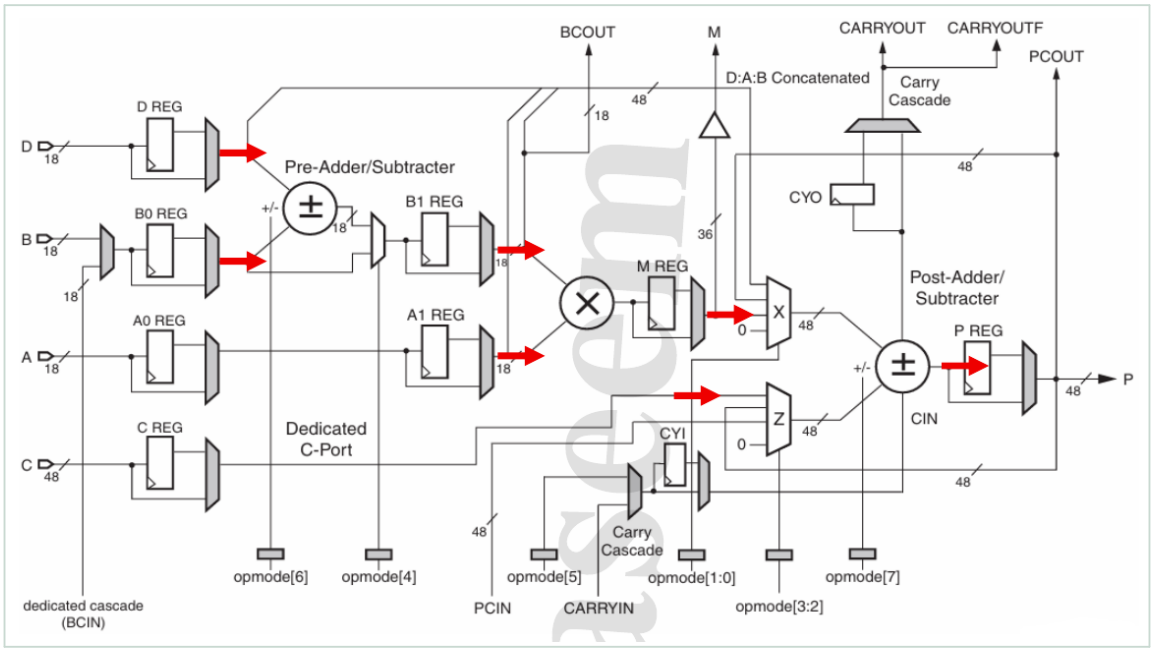



*Figure 2.1: Path 1 data flow (source: testbench description).*

## 2.3 Verify DSP path 2: pre-add → multiply, post-adder idle

**OPMODE = 8'b00010000.** Only the pre-adder is enabled (D + B = 35 = 0x23); X and Z are forced to zero, so P = 0. The multiplier output is M = 35 × 20 = 700 (0x2BC).

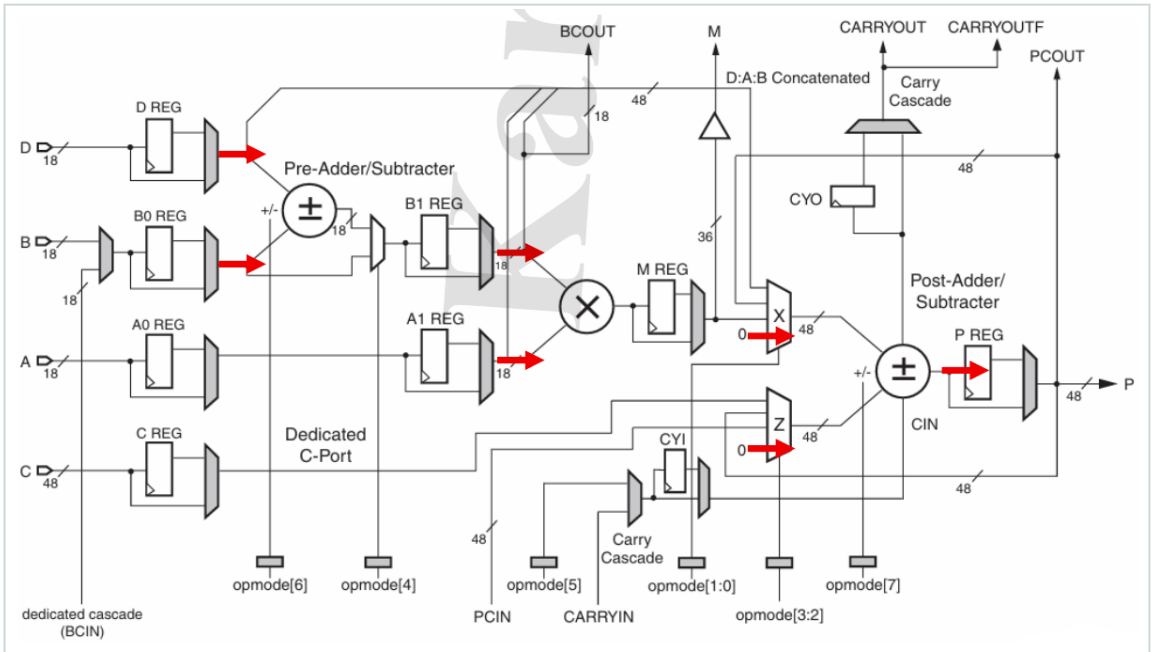



*Figure 2.2: Path 2 data flow (source: testbench description).*

## 2.4 Verify DSP path 3: P feedback (accumulate)

**OPMODE = 8'b00001010.** The pre-adder is bypassed, so B passes straight through (BCOUT = 0xA) and M = 10 × 20 = 200 (0xC8). Both X and Z take P, so P is fed back into the post-adder. Because P depends on its own previous value, the testbench checks that P = PCOUT and CARRYOUT = CARRYOUTF (feedback consistency).

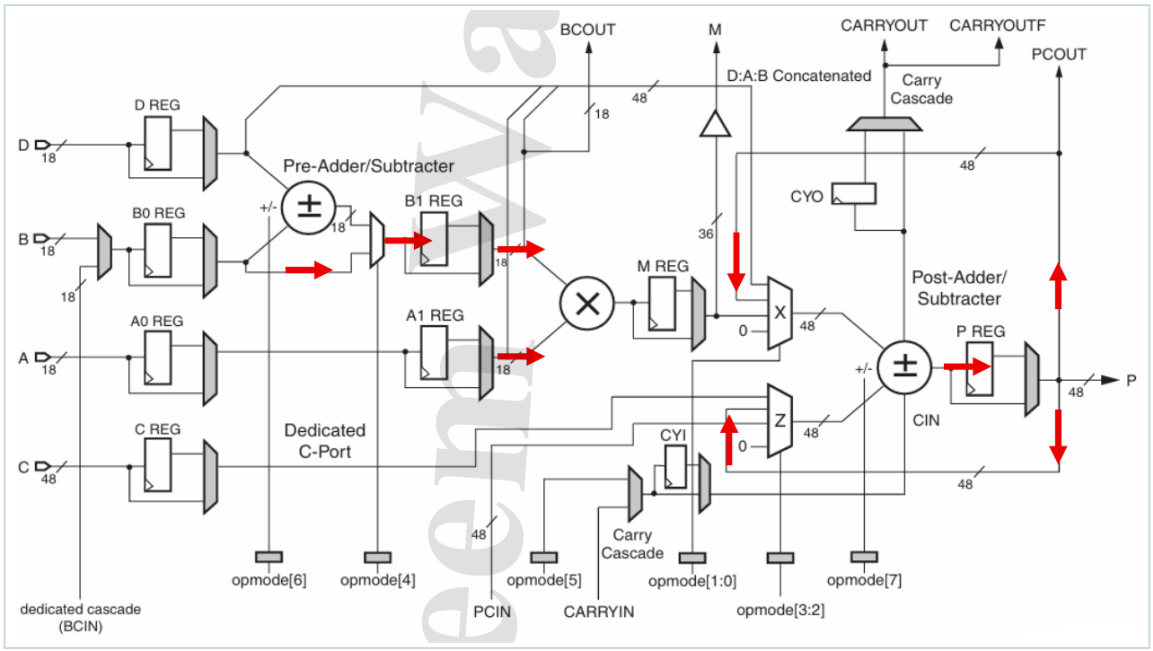



*Figure 2.3: Path 3 data flow (source: testbench description).*

## 2.5 Verify DSP path 4: D:A:B and PCIN with post-subtract

**OPMODE = 8'b10100111.** X receives the concatenated D:A:B bus, Z receives PCIN, the post-adder subtracts and OPMODE[5] = 1 supplies the carry-in. With A = 5 and B = 6, BCOUT = 6 and M = 30 (0x1E). The 48-bit result is 0xFE6FFFEC0BB1 with CARRYOUT = 1.

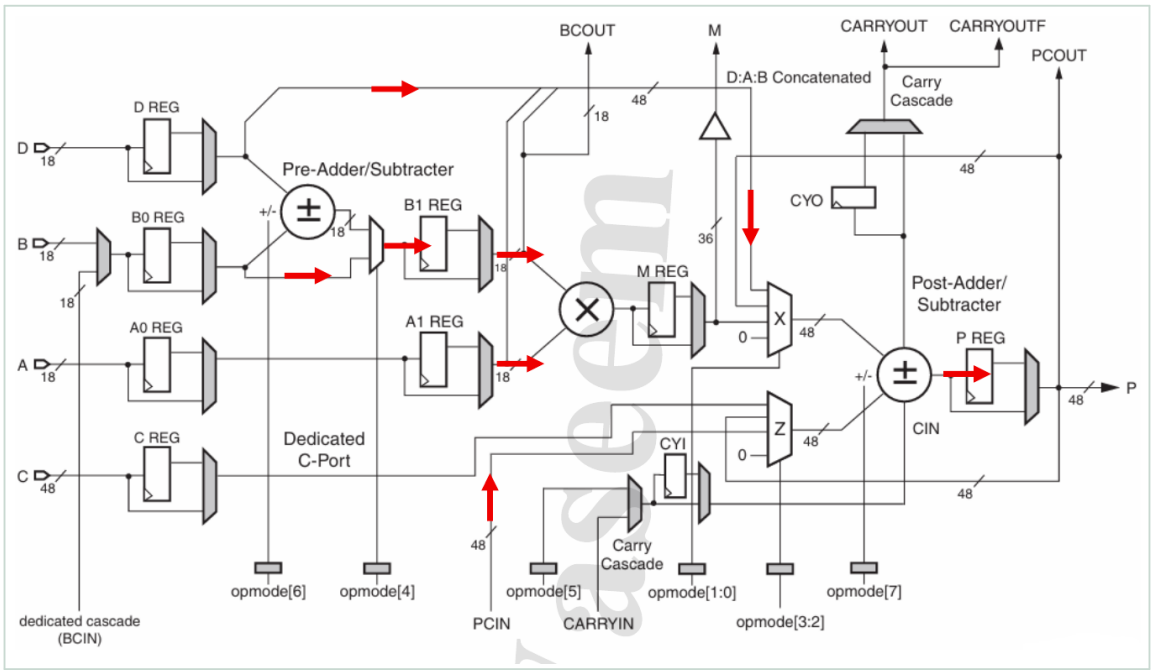



*Figure 2.4: Path 4 data flow (source: testbench description).*

## 2.6 Testbench source code

The testbench is shown in six consecutive screenshots. It instantiates the DUT with default parameters, generates a 10 ns clock, defines two checking tasks (check48 for buses and check1 for single bits) and ends with a pass/fail summary.

### Part 1: declarations

```
1  // DSP48A1 Testbench - Directed / self-checking test patterns
2  `timescale 1ns/1ps
3
4  module DSP48A1_tb;
5
6      // Clock
7      reg CLK;
8
9      // Data inputs
10     reg [17:0] A, B, D;
11     reg [47:0] C;
12     reg        CARRYIN;
13     reg [7:0]  OPMODE;
14     reg [17:0] BCIN;
15     reg [47:0] PCIN;
16
17     // Reset inputs
18     reg RSTA, RSTB, RSTM, RSTP, RSTC, RSTD, RSTCARRYIN, RSTOPMODE;
19
20     // Clock-enable inputs
21     reg CEA, CEB, CEM, CEP, CEC, CED, CECARRYIN, CEOPMODE;
22
23     // Outputs
24     wire [17:0] BCOUT;
25     wire [47:0] PCOUT;
26     wire [47:0] P;
27     wire [35:0] M;
28     wire        CARRYOUT;
29     wire        CARRYOUTF;
30
31     integer pass_count = 0;
32     integer fail_count = 0;
33
34
```

*Listing 2.1: dsp48a1_tb_.v, lines 1–34*

## Part 2: DUT instance, clock and check48 task

```
35      // DUT instantiation - default parameters match the spec:
36      // A0REG=0 A1REG=1 B0REG=0 B1REG=1 CREG=1 DREG=1 MREG=1 PREG=1
37      // CARRYINREG=1 CARRYOUTREG=1 OPMODEREG=1
38      // CARRYINSEL="OPMODE5" B_INPUT="DIRECT" RSTTYPE="SYNC"
39      DSP48A1 dut (
40          .A(A), .B(B), .D(D), .C(C),
41          .CLK(CLK), .CARRYIN(CARRYIN),
42          .OPMODE(OPMODE), .BCIN(BCIN), .PCIN(PCIN),
43          .RSTA(RSTA), .RSTB(RSTB), .RSTM(RSTM), .RSTP(RSTP),
44          .RSTC(RSTC), .RSTD(RSTD), .RSTCARRYIN(RSTCARRYIN), .RSTOPMODE(RSTOPMODE),
45          .CEA(CEA), .CEB(CEB), .CEM(CEM), .CEP(CEP),
46          .CEC(CEC), .CED(CED), .CECARRYIN(CECARRYIN), .CEOPMODE(CEOPMODE),
47          .BCOUT(BCOUT), .PCOUT(PCOUT), .P(P), .M(M),
48          .CARRYOUT(CARRYOUT), .CARRYOUTF(CARRYOUTF)
49      );
50
51      // Clock: 10 ns period
52      initial CLK = 1'b0;
53      always #5 CLK = ~CLK;
54
55      // Self-check helper task
56      task check48(input [47:0] actual, input [47:0] expected, input [255:0] name);
57          begin
58              if (actual === expected) begin
59                  pass_count = pass_count + 1;
60                  $display("[PASS] %s : expected=%h actual=%h", name, expected, actual);
61              end else begin
62                  fail_count = fail_count + 1;
63                  $display("[FAIL] %s : expected=%h actual=%h", name, expected, actual);
64              end
65          end
66      endtask
67
```

*Listing 2.2: dsp48a1_tb_.v, lines 35–67*

## Part 3: check1 task and reset test (2.1)

```
68    task check1(input actual, input expected, input [255:0] name);
69        begin
70            if (actual === expected) begin
71                pass_count = pass_count + 1;
72                $display("[PASS] %s : expected=%b actual=%b", name, expected, actual);
73            end else begin
74                fail_count = fail_count + 1;
75                $display("[FAIL] %s : expected=%b actual=%b", name, expected, actual);
76            end
77        end
78    endtask
79
80    // Stimulus
81    initial begin
82        // 2.1 Verify Reset Operation
83        RSTA=1; RSTB=1; RSTM=1; RSTP=1; RSTC=1; RSTD=1; RSTCARRYIN=1; RSTOPMODE=1;
84        CEA=0; CEB=0; CEM=0; CEP=0; CEC=0; CED=0; CECARRYIN=0; CEOPMODE=0;
85
86        A = $random; B = $random; D = $random; C = {$random,$random};
87        CARRYIN = $random; OPMODE = $random; BCIN = $random; PCIN = {$random,$random};
88
89        @(negedge CLK);
90        #1;
91        $display("\n---- Reset Check ----");
92        check48(P,     48'h0, "Reset: P");
93        check48(PCOUT, 48'h0, "Reset: PCOUT");
94        check48({30'b0,BCOUT}, 48'h0, "Reset: BCOUT");
95        check48({12'b0,M},     48'h0, "Reset: M");
96        check1(CARRYOUT,  1'b0, "Reset: CARRYOUT");
97        check1(CARRYOUTF, 1'b0, "Reset: CARRYOUTF");
98
99        RSTA=0; RSTB=0; RSTM=0; RSTP=0; RSTC=0; RSTD=0; RSTCARRYIN=0; RSTOPMODE=0;
100       CEA=1; CEB=1; CEM=1; CEP=1; CEC=1; CED=1; CECARRYIN=1; CEOPMODE=1;
101
```

*Listing 2.3: dsp48a1_tb_.v, lines 68–101*

## Part 4: paths 1 and 2 (2.2 and 2.3)

```
102         // 2.2 Verify DSP Path 1 : pre-sub -> mult -> post-sub
103         // OPMODE = 8'b11011101
104         @(negedge CLK);
105         OPMODE = 8'b11011101;
106         A = 20; B = 10; C = 350; D = 25;
107         BCIN = $random; PCIN = {$random,$random}; CARRYIN = $random;
108
109         repeat (4) @(negedge CLK);
110         #1;
111         $display("\n---- Path 1 Check ----");
112         check48({30'b0,BCOUT}, 48'h0000000000f, "Path1: BCOUT");
113         check48({12'b0,M},     48'h00000000012c, "Path1: M");
114         check48(P,     48'h32, "Path1: P");
115         check48(PCOUT, 48'h32, "Path1: PCOUT");
116         check1(CARRYOUT,  1'b0, "Path1: CARRYOUT");
117         check1(CARRYOUTF, 1'b0, "Path1: CARRYOUTF");
118
119         // 2.3 Verify DSP Path 2 : pre-add -> mult -> post-add(zeros)
120         // OPMODE = 8'b00010000
121         @(negedge CLK);
122         OPMODE = 8'b00010000;
123         A = 20; B = 10; C = 350; D = 25;
124         BCIN = $random; PCIN = {$random,$random}; CARRYIN = $random;
125
126         repeat (3) @(negedge CLK);
127         #1;
128         $display("\n---- Path 2 Check ----");
129         check48({30'b0,BCOUT}, 48'h023, "Path2: BCOUT");
130         check48({12'b0,M},     48'h2bc, "Path2: M");
131         check48(P,     48'h0, "Path2: P");
132         check48(PCOUT, 48'h0, "Path2: PCOUT");
133         check1(CARRYOUT,  1'b0, "Path2: CARRYOUT");
134         check1(CARRYOUTF, 1'b0, "Path2: CARRYOUTF");
135
```

*Listing 2.4: dsp48a1_tb_.v, lines 102–135*

## Part 5: paths 3 and 4 (2.4 and 2.5)

```
136         // 2.4 Verify DSP Path 3 : no pre-adder, P feedback (accumulator)
137         // OPMODE = 8'b00001010
138         @(negedge CLK);
139         OPMODE = 8'b00001010;
140         A = 20; B = 10; C = 350; D = 25;
141         BCIN = $random; PCIN = {$random,$random}; CARRYIN = $random;
142
143         repeat (3) @(negedge CLK);
144         #1;
145         $display("\n---- Path 3 Check ----");
146         check48({30'b0,BCOUT}, 48'h00a, "Path3: BCOUT");
147         check48({12'b0,M},     48'h0c8, "Path3: M");
148         // P/PCOUT and CARRYOUT/CARRYOUTF should equal each other
149         // (P is fed back through X and Z muxes)
150         check48(P, PCOUT, "Path3: P == PCOUT (feedback consistency)");
151         check1(CARRYOUT, CARRYOUTF, "Path3: CARRYOUT == CARRYOUTF (feedback consistency)");
152
153         // 2.5 Verify DSP Path 4 : D:A:B concat -> X , PCIN -> Z , subtract
154         // OPMODE = 8'b10100111
155         @(negedge CLK);
156         OPMODE = 8'b10100111;
157         A = 5; B = 6; C = 350; D = 25; PCIN = 3000;
158         BCIN = $random; CARRYIN = $random;
159
160         repeat (3) @(negedge CLK);
161         #1;
162         $display("\n---- Path 4 Check ----");
163         check48({30'b0,BCOUT}, 48'h006, "Path4: BCOUT");
164         check48({12'b0,M},     48'h01e, "Path4: M");
165         check48(P,     48'hfe6fffec0bb1, "Path4: P");
166         check48(PCOUT, 48'hfe6fffec0bb1, "Path4: PCOUT");
167         check1(CARRYOUT,  1'b1, "Path4: CARRYOUT");
168         check1(CARRYOUTF, 1'b1, "Path4: CARRYOUTF");
169
```

*Listing 2.5: dsp48a1_tb_.v, lines 136–169*

## Part 6: summary and end of simulation

```
170         // Summary
171         #20;
172         $display("\n====================================================");
173         $display("TEST SUMMARY : %0d PASSED , %0d FAILED", pass_count, fail_count);
174         $display("====================================================");
175
176         $stop;
177     end
178
179 endmodule
180
```

*Listing 2.6: dsp48a1_tb_.v, lines 170–180*

# 3 Do File

QuestaSim script (run_.do)

The script recreates the `work` library, compiles the submodules first, then the top level and the testbench, loads the simulation with full visibility (`+acc`), adds the waves grouped under dividers (inputs, resets, clock enables, outputs, internal signals), and runs to completion.

```
# Create / map work library
if [file exists work] {
    vdel -all -lib work
}
vlib work
vmap work work

# Compile submodules first
vlog -sv pipe_reg.v
vlog -sv mux4.v
vlog -sv preadder_sub.v
vlog -sv postadder_sub.v

# Compile top-level DSP48A1
vlog -sv dsp48a1_top.v

# Compile testbench
vlog -sv dsp48a1_tb_.v

# Load the design
vsim -voptargs=+acc work.DSP48A1_tb

# Add all signals to the waveform window
add wave -radix hex sim:/DSP48A1_tb/CLK
add wave -radix hex sim:/DSP48A1_tb/OPMODE
add wave -radix decimal sim:/DSP48A1_tb/A
add wave -radix decimal sim:/DSP48A1_tb/B
add wave -radix decimal sim:/DSP48A1_tb/C
add wave -radix decimal sim:/DSP48A1_tb/D
add wave -radix hex sim:/DSP48A1_tb/BCIN
add wave -radix hex sim:/DSP48A1_tb/PCIN
add wave -radix hex sim:/DSP48A1_tb/CARRYIN

add wave -divider "Resets"
add wave sim:/DSP48A1_tb/RSTA
add wave sim:/DSP48A1_tb/RSTB
add wave sim:/DSP48A1_tb/RSTC
add wave sim:/DSP48A1_tb/RSTD
add wave sim:/DSP48A1_tb/RSTM
add wave sim:/DSP48A1_tb/RSTP
add wave sim:/DSP48A1_tb/RSTCARRYIN
add wave sim:/DSP48A1_tb/RSTOPMODE
```

*Listing 3.1: run_.do, part 1: library setup, compilation, input and reset waves*

```
add wave -divider "Clock Enables"
add wave sim:/DSP48A1_tb/CEA
add wave sim:/DSP48A1_tb/CEB
add wave sim:/DSP48A1_tb/CEC
add wave sim:/DSP48A1_tb/CED
add wave sim:/DSP48A1_tb/CEM
add wave sim:/DSP48A1_tb/CEP
add wave sim:/DSP48A1_tb/CECARRYIN
add wave sim:/DSP48A1_tb/CEOPMODE

add wave -divider "Outputs"
add wave -radix hex sim:/DSP48A1_tb/BCOUT
add wave -radix hex sim:/DSP48A1_tb/PCOUT
add wave -radix hex sim:/DSP48A1_tb/P
add wave -radix hex sim:/DSP48A1_tb/M
add wave sim:/DSP48A1_tb/CARRYOUT
add wave sim:/DSP48A1_tb/CARRYOUTF

# ------------------------------------------------------------------
# Internal signal

add wave -divider "Internal DUT signals (through submodule instances)"
add wave -radix hex sim:/DSP48A1_tb/dut/d_reg_out
add wave -radix hex sim:/DSP48A1_tb/dut/b0_reg_out
add wave -radix hex sim:/DSP48A1_tb/dut/b1_reg_out
add wave -radix hex sim:/DSP48A1_tb/dut/a1_reg_out
add wave -radix hex sim:/DSP48A1_tb/dut/u_mreg/Q
add wave -radix hex sim:/DSP48A1_tb/dut/opmode_use
add wave -radix hex sim:/DSP48A1_tb/dut/x_val
add wave -radix hex sim:/DSP48A1_tb/dut/z_val
add wave -radix hex sim:/DSP48A1_tb/dut/p_reg_out


run -all


wave zoom full
```

*Listing 3.2: run_.do, part 2: clock enables, outputs, internal signals, run*

### Simulation transcript

The testbench ends with a summary line from the self-checking tasks: all 28 comparisons pass and none fail.

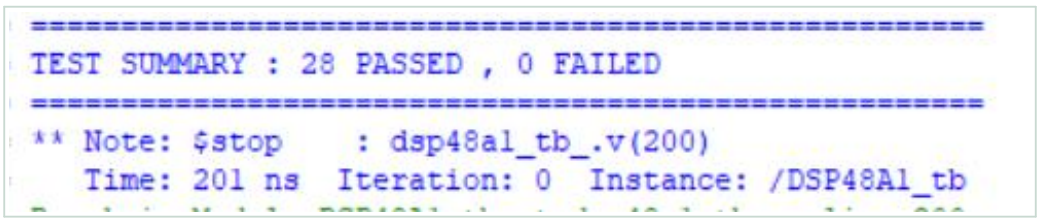



*Figure 3.1: QuestaSim transcript, test summary*

# 4 QuestaSim Snippets

Waveforms with input stimulus and output values visible

The waveform is shown in four views. Inputs and outputs are labelled and the radix is chosen per signal. The first view is the whole run; the second zooms into the DSP path tests; the last two include the internal DUT signals (register outputs, X and Z mux outputs and the P register).

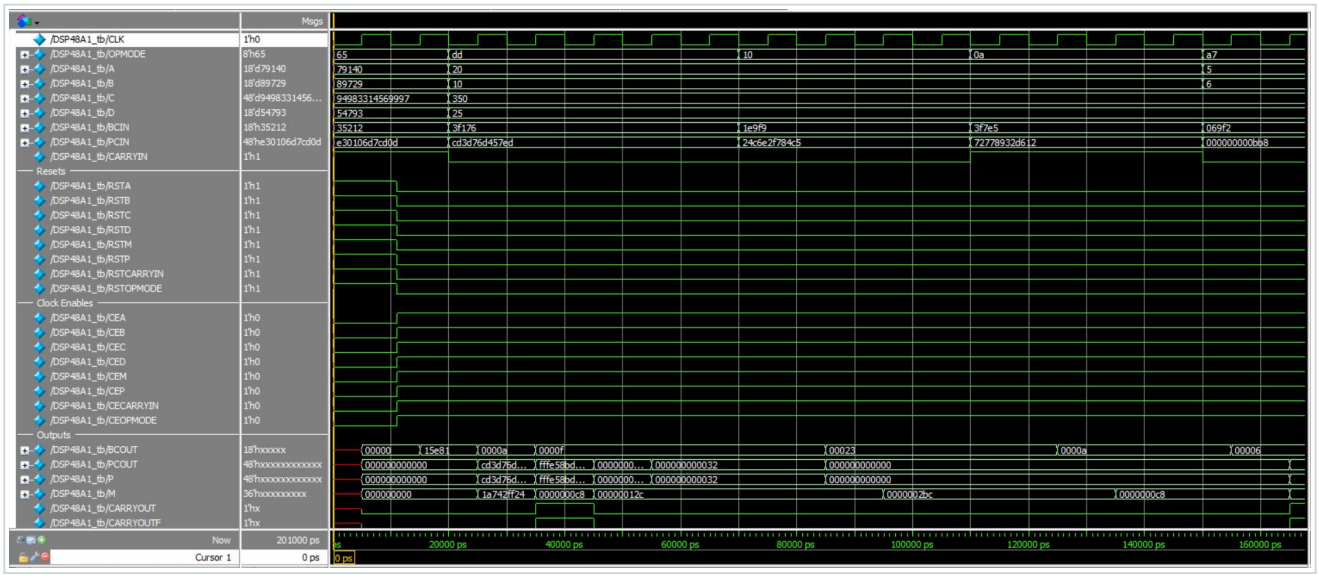


*Figure 4.1: Full simulation run, inputs, resets, clock enables and outputs*

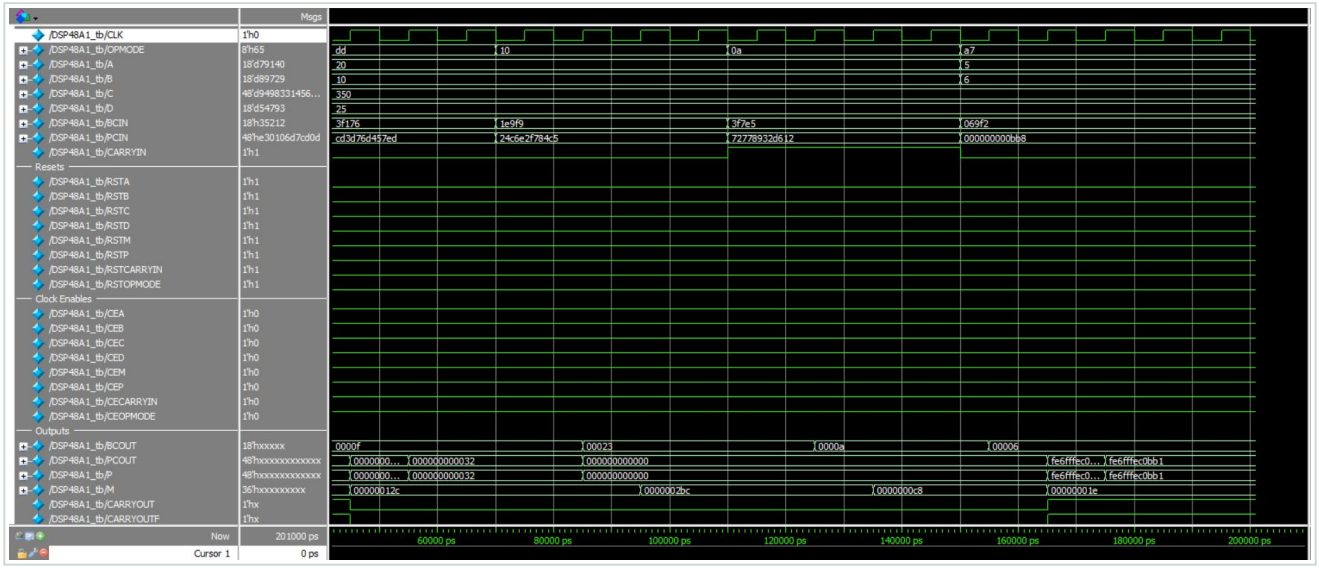


*Figure 4.2: DSP path tests (zoomed view)*

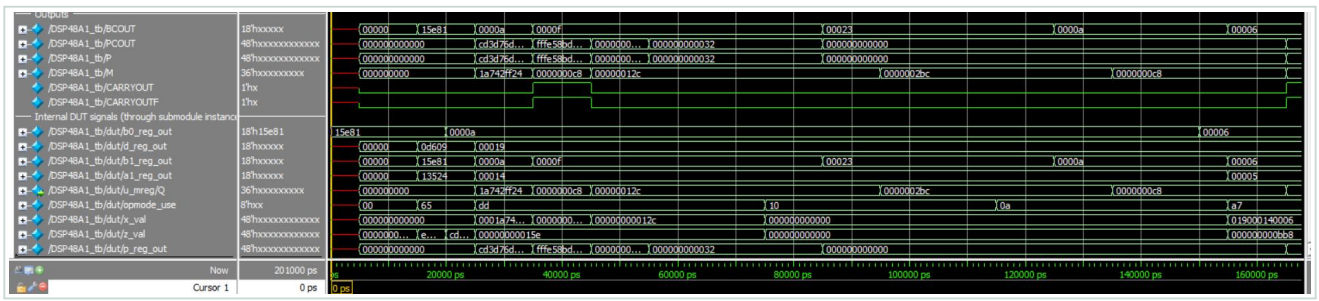


*Figure 4.3: Outputs together with internal DUT signals (part 1)*

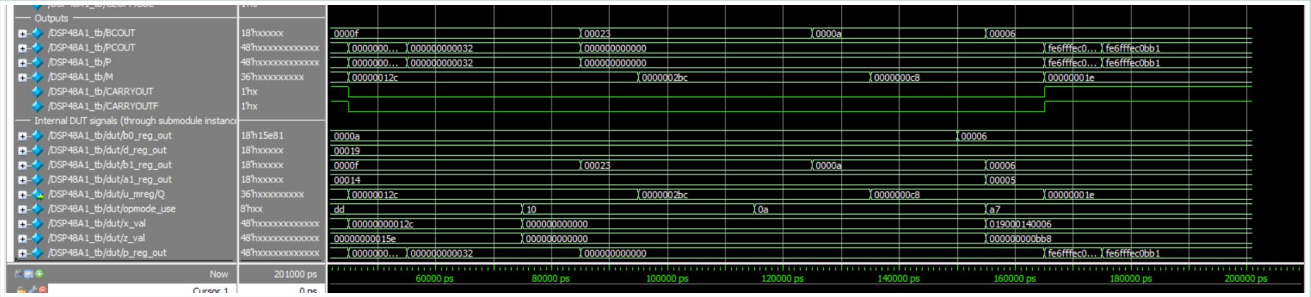



*Figure 4.4: Outputs together with internal DUT signals (part 2)*

# 5 Constraint File

Timing constraint only: 100 MHz clock on pin W5

The XDC file defines only the clock, as required: the CLK port is assigned to pin W5 and a 10 ns period (100 MHz, 50 % duty cycle) is created. The rest of the Basys-3 template is left commented out, because the part `xc7a200tffg1156-3` is targeted directly and the design has far more ports than the board offers.

constraints_1.xdc

D: > wass_projects > project_1_af > constraints_1.xdc

```
1
2  ## Constraint file - DSP48A1 Project
3
4  ## Clock signal
5  set_property -dict { PACKAGE_PIN W5   IOSTANDARD LVCMOS33 } [get_ports CLK]
6  create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports CLK]
7
8
```

*Listing 5.1: constraints_1.xdc*

# 6 Elaboration

"Messages" tab and schematic snippets

The elaborated schematic shows the RTL hierarchy as 17 cells and 346 I/O ports: the pipeline registers, the two muxes, the pre- and post-adders and the multiplier.

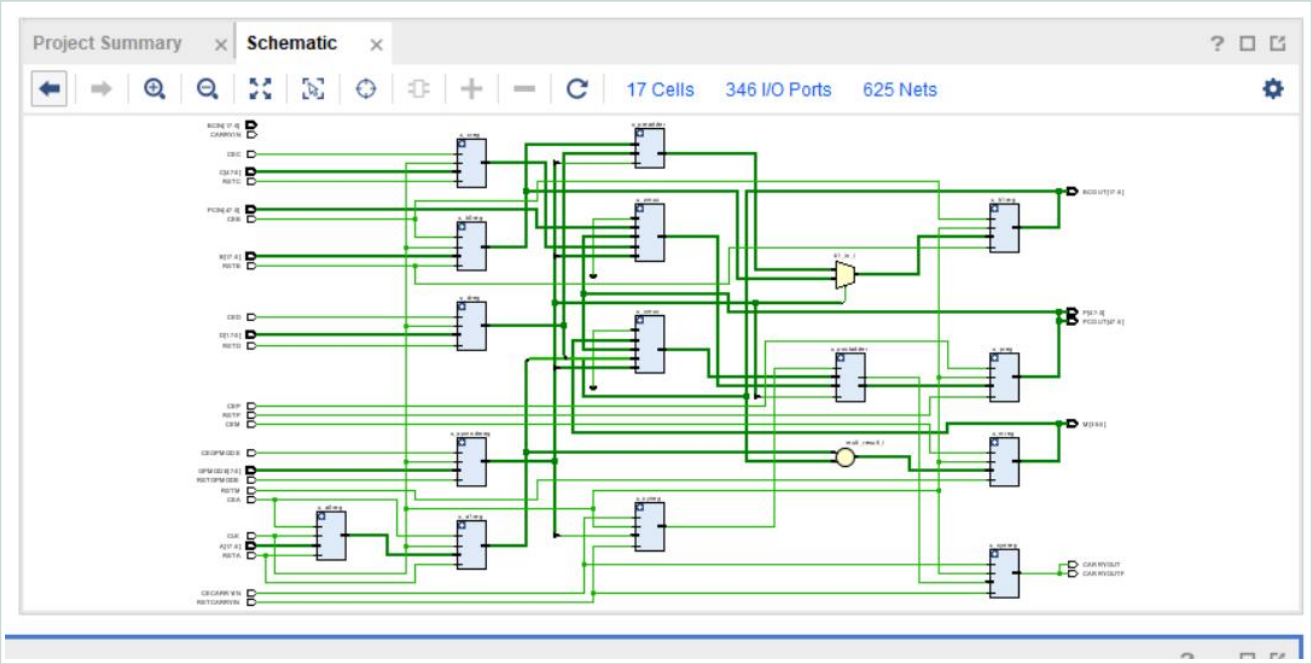



*Figure 6.1: Elaborated design schematic*

### Messages tab

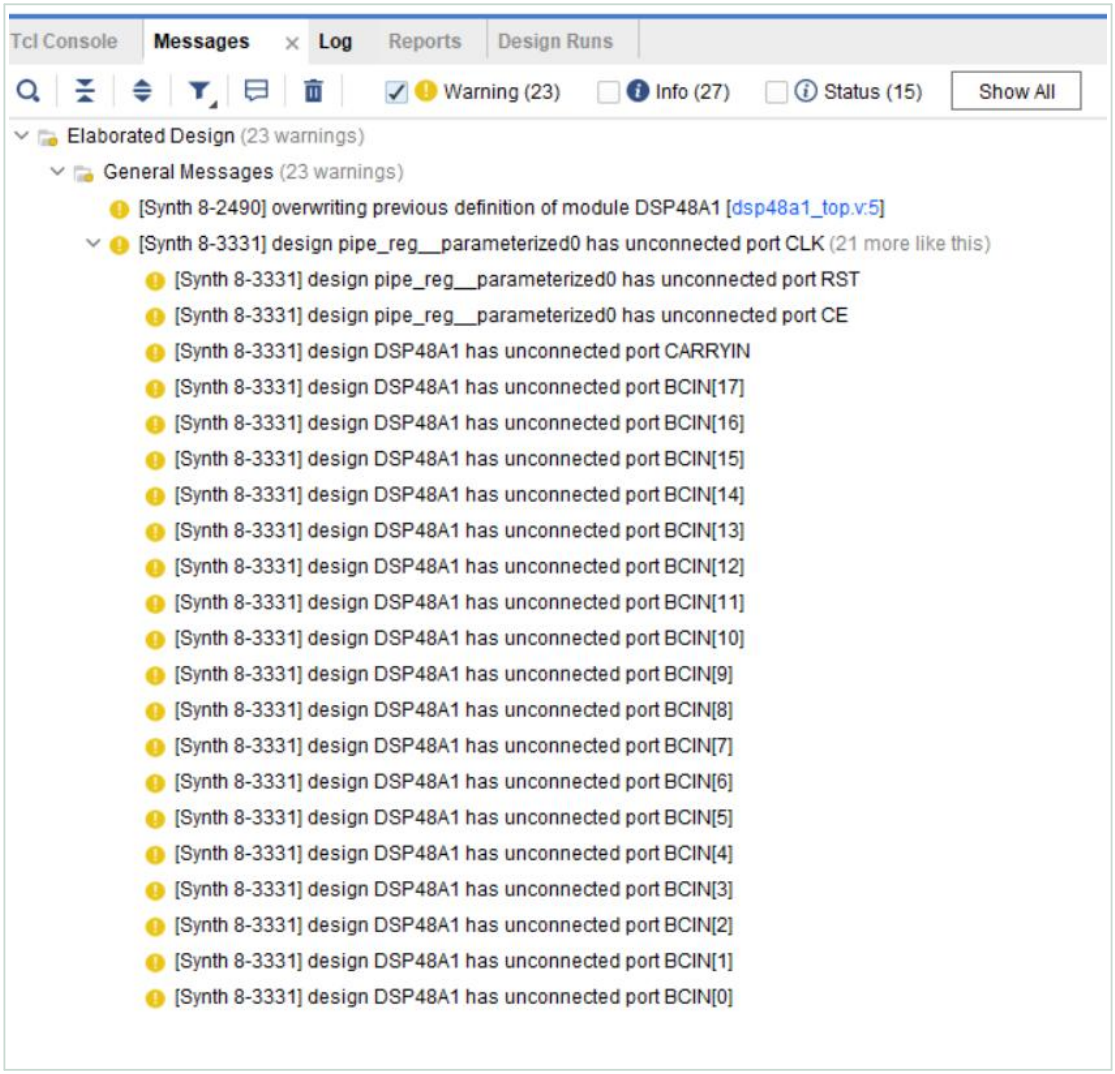



*Figure 6.2: Elaboration messages (23 warnings, no errors or critical warnings)*

**Why these warnings appear.** All of them are unused-port warnings that follow directly from the default parameter values: BCIN is unconnected because B_INPUT = "DIRECT", CARRYIN is unconnected because CARRYINSEL = "OPMODE5", and the CLK/RST/CE ports of the bypassed registers (A0REG = B0REG = 0) are unused because those stages are plain wires.

# 7 Synthesis

"Messages" tab, utilization report, timing report and schematic snippets

## 7.1 Synthesized schematic

After synthesis the multiplier and post-adder are mapped to the dedicated DSP48E1 primitive. Synthesis reports 341 cells, 346 I/O ports and 1041 nets.

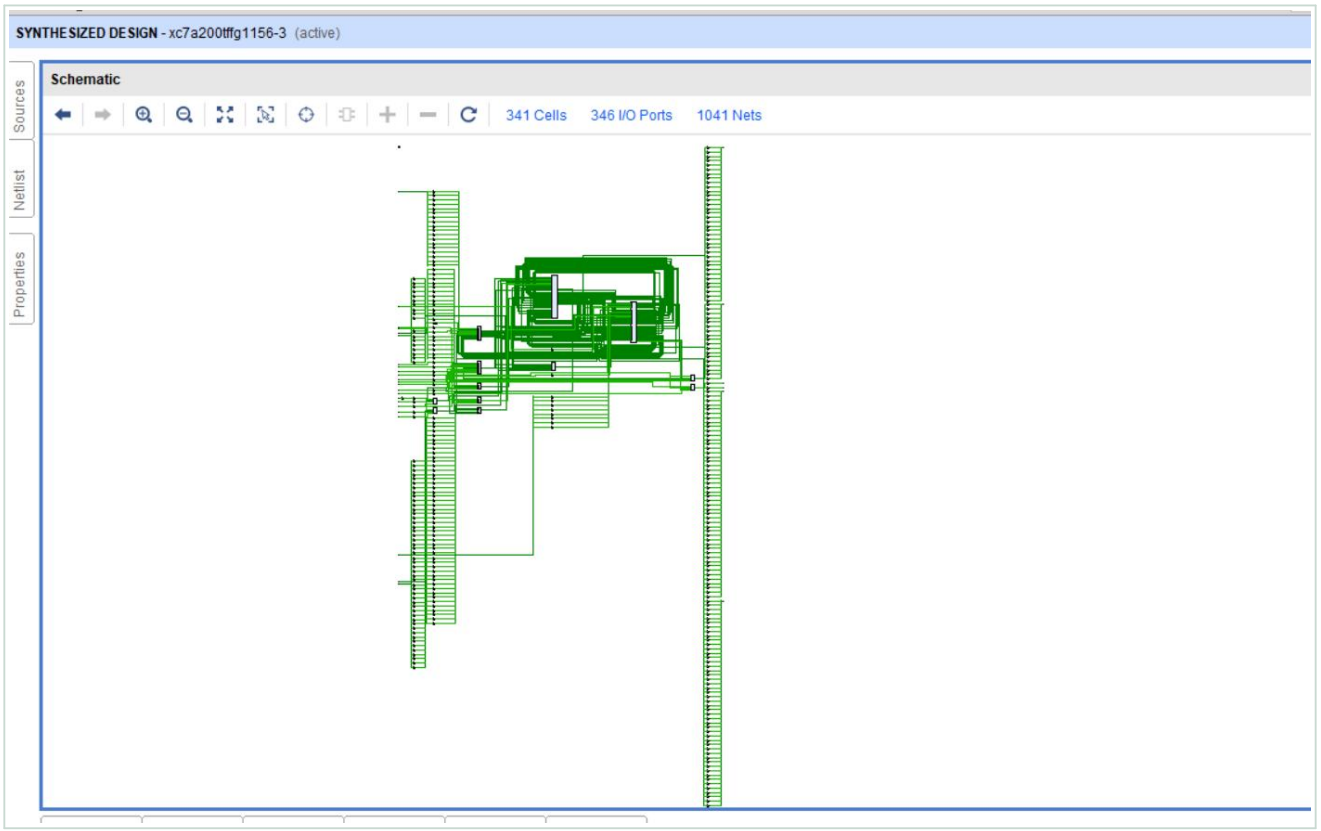



*Figure 7.1: Synthesized schematic (xc7a200tffg1156-3)*

## 7.2 Messages tab

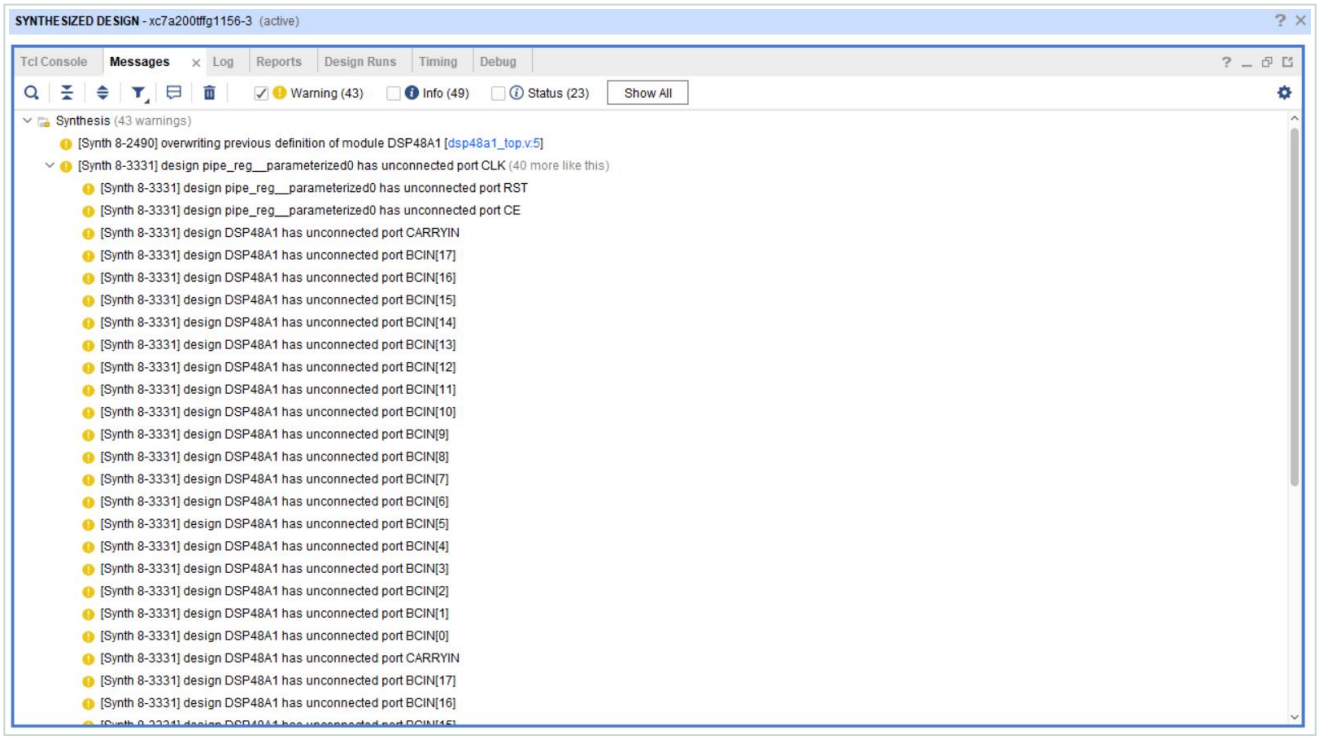



*Figure 7.2: Synthesis messages (43 warnings)*

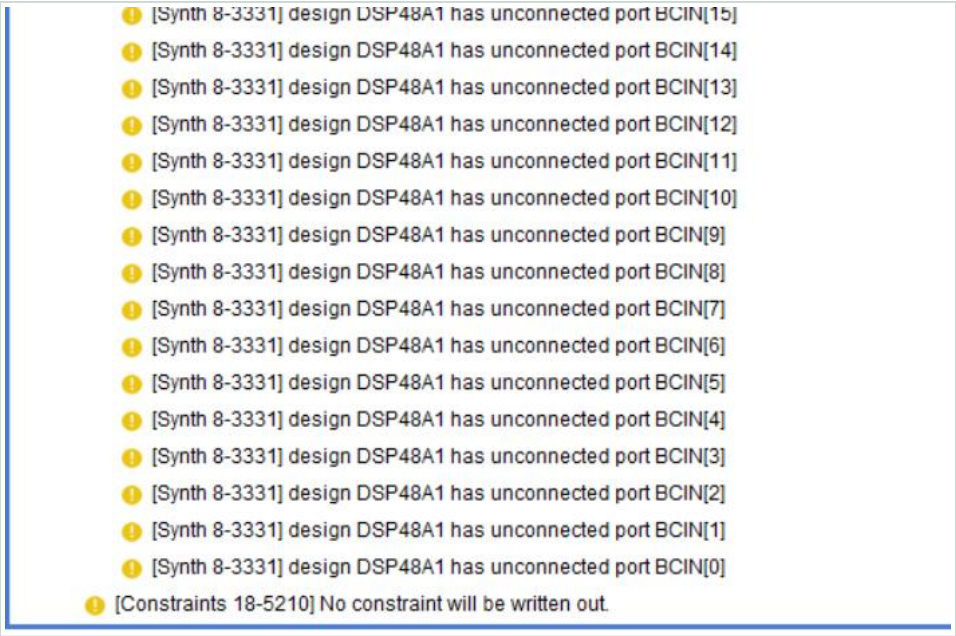



*Figure 7.3: Synthesis messages, continued*

## 7.3 Timing report

Post-synthesis timing at 100 MHz: worst negative slack +5.216 ns, worst hold slack +0.182 ns and worst pulse-width slack +4.500 ns. There are no failing endpoints and all user-specified constraints are met.

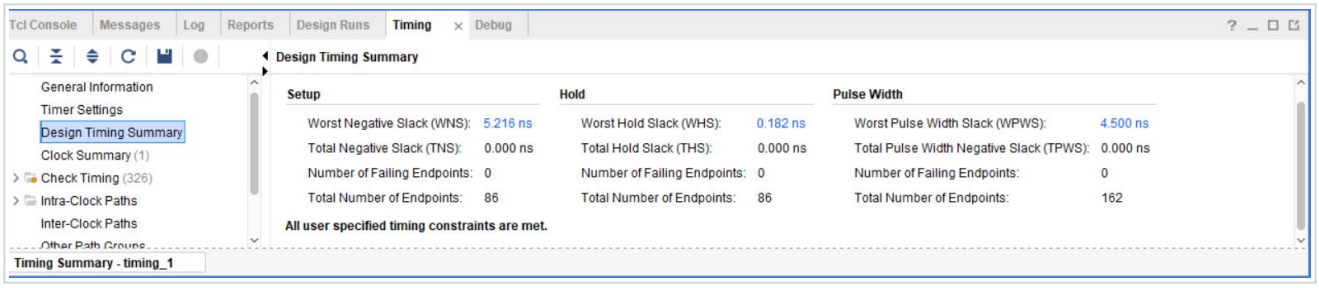



*Figure 7.4: Design timing summary after synthesis*

## 7.4 Utilization report

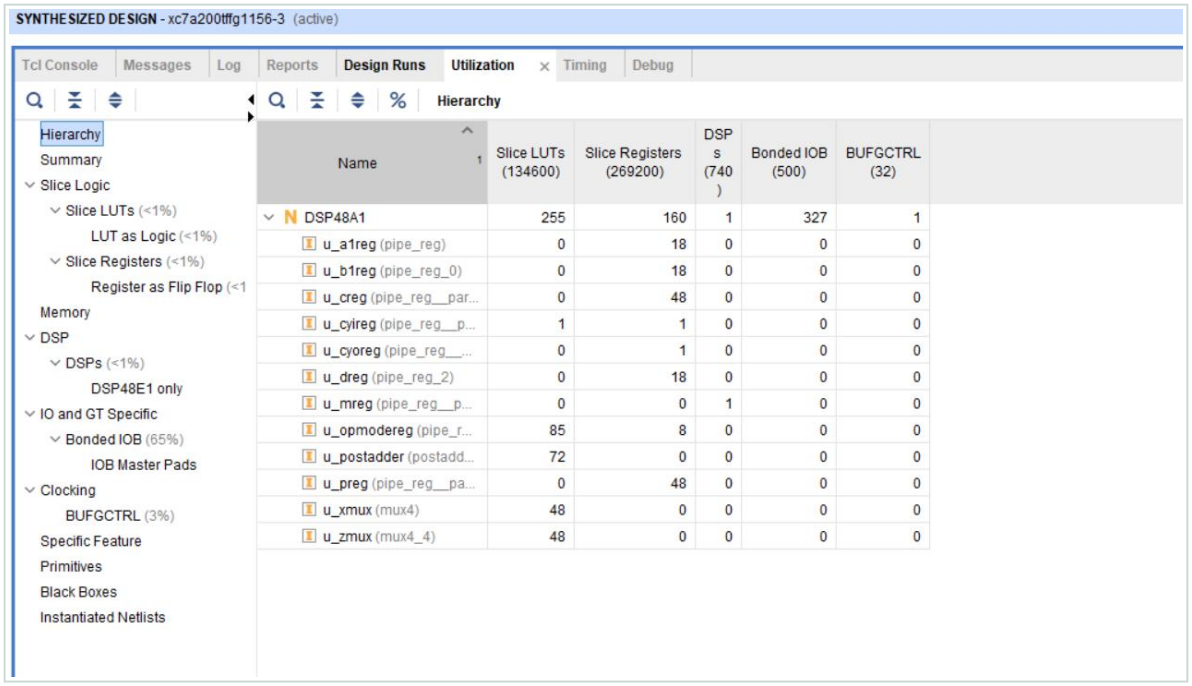



*Figure 7.5: Utilization by hierarchy after synthesis*

### Utilization summary after synthesis

| Resource | Used | Available | Utilization |
|---|---|---|---|
| LUT | 255 | 134,600 | 0.19 % |
| FF | 160 | 269,200 | 0.06 % |
| DSP | 1 | 740 | 0.14 % |
| IO | 327 | 500 | 65.40 % |

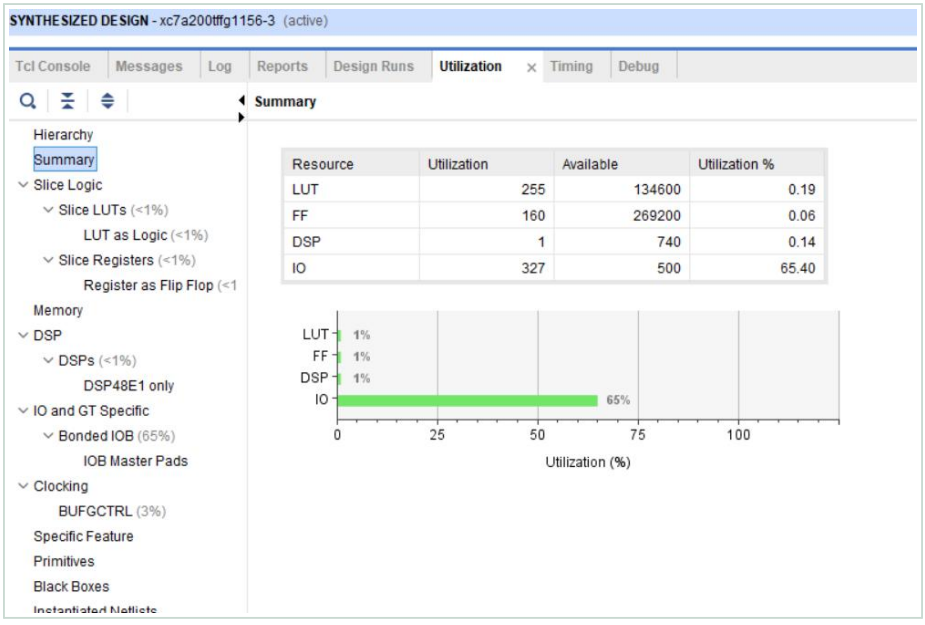



*Figure 7.6: Utilization summary after synthesis*

# 8 Implementation

"Messages" tab, utilization report, timing report and device snippets

## 8.1 Device view

The placed design uses a single DSP48E1 site plus a small amount of logic, located in one clock region of the device.

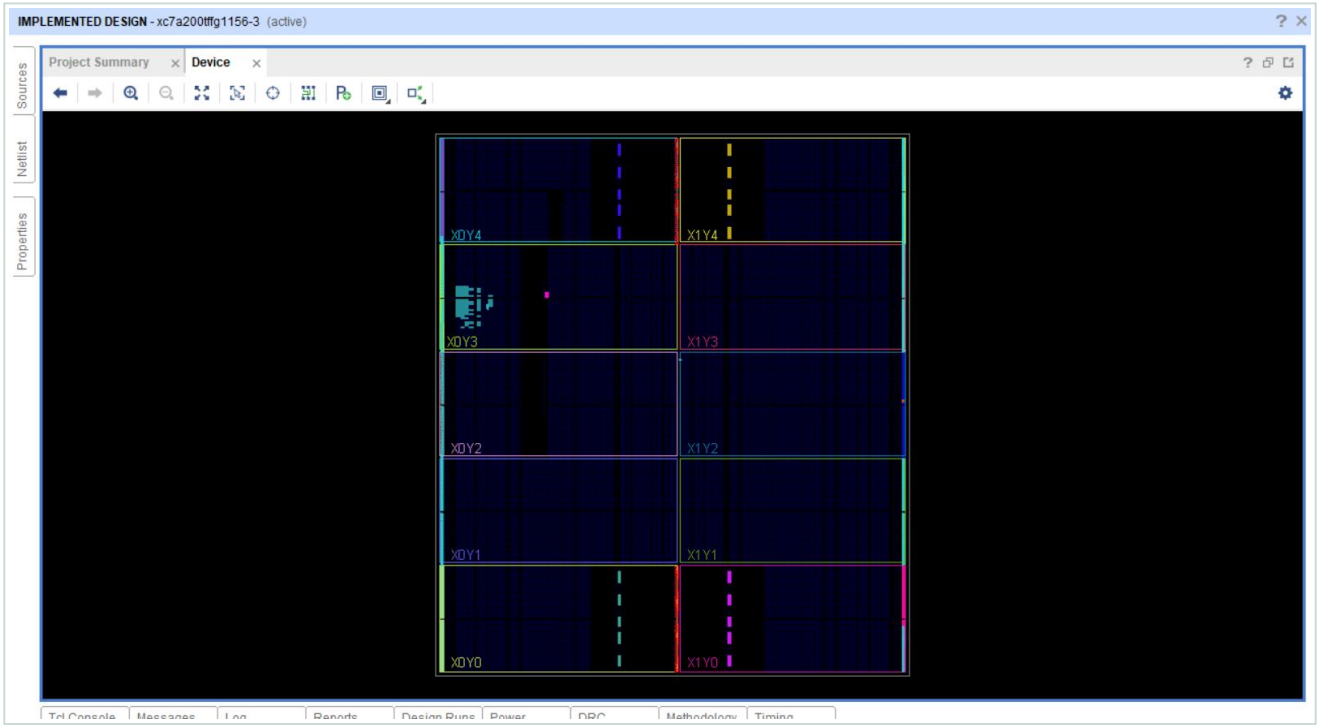



*Figure 8.1: Implemented design, device view*

## 8.2 Messages tab




*Figure 8.2: Implementation messages (43 warnings, no errors or critical warnings)*

## 8.3 Timing report

After place and route at 100 MHz: worst negative slack +3.696 ns, worst hold slack +0.271 ns and worst pulse-width slack +4.500 ns. No failing endpoints; all user-specified timing constraints are met.

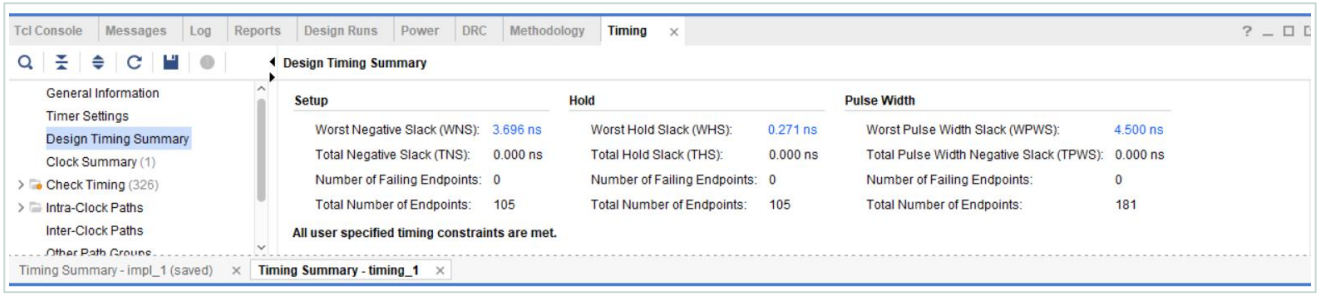



*Figure 8.3: Design timing summary after implementation*

## 8.4 Utilization report

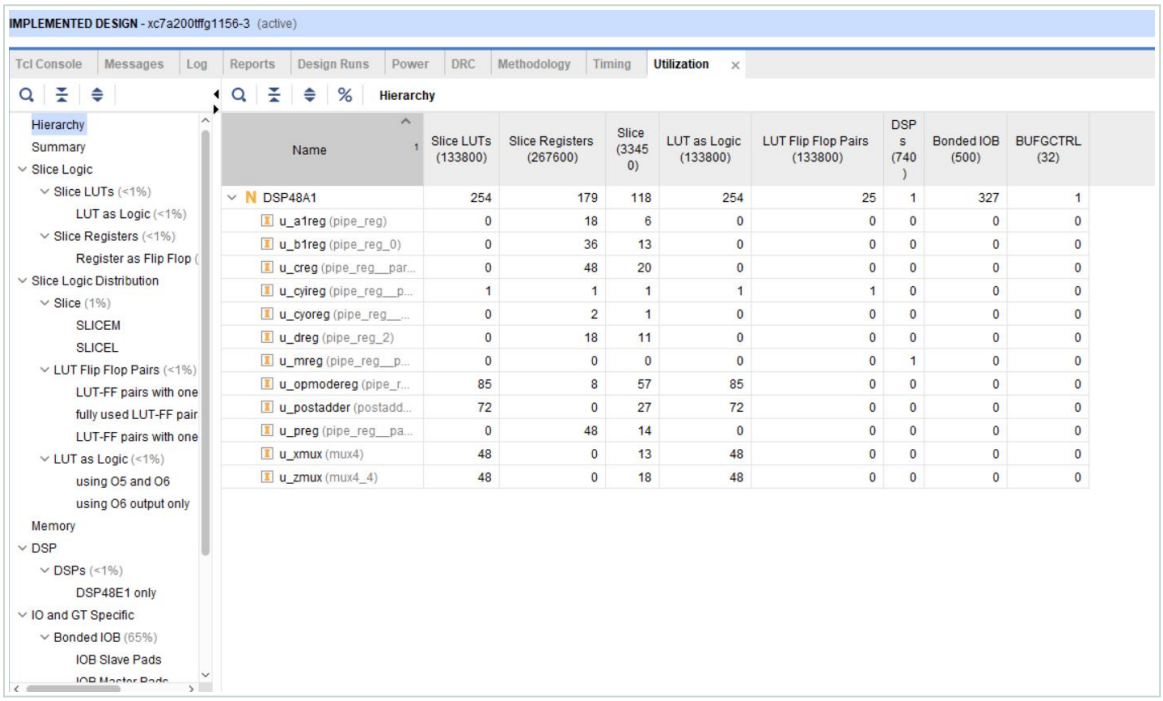



*Figure 8.4: Utilization by hierarchy after implementation*

## Utilization summary after implementation

| Resource | Used | Available | Utilization |
|---|---|---|---|
| LUT | 254 | 133,800 | 0.19 % |
| FF | 179 | 267,600 | 0.07 % |
| DSP | 1 | 740 | 0.14 % |
| IO | 327 | 500 | 65.40 % |

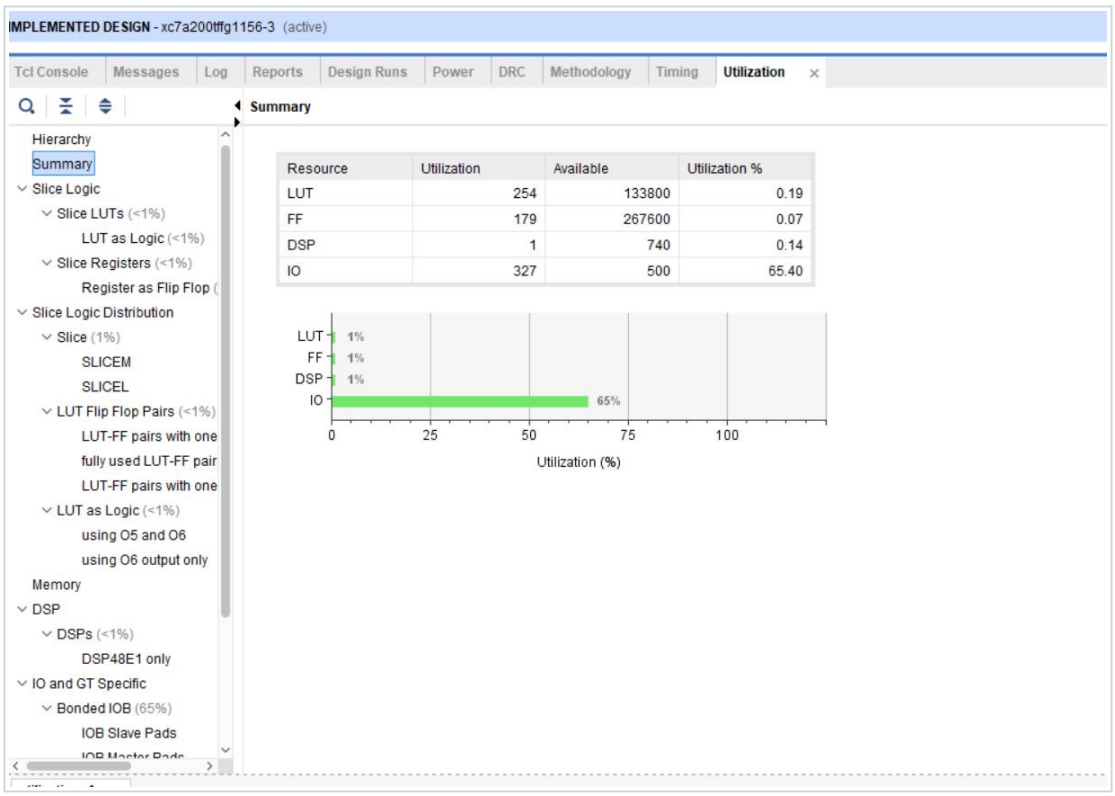



*Figure 8.5: Utilization summary after implementation*

| Metric | Synthesis | Implementation |
|---|---|---|
| Worst negative slack (setup) | +5.216 ns | +3.696 ns |
| Worst hold slack | +0.182 ns | +0.271 ns |
| Worst pulse-width slack | +4.500 ns | +4.500 ns |
| LUT / FF / DSP / IO | 255 / 160 / 1 / 327 | 254 / 179 / 1 / 327 |

Setup slack is smaller after implementation because real routing delays replace the synthesis estimates; it is still positive.

# 9 Linting

Questa Lint with the default methodology and goals

The lint report for DSP48A1 shows **0 errors and 0 warnings** and a design quality score of 100 %. The only findings are 5 informational/style items (a constant condition, a repeated parameter name across modules, and several ports on one line). The first screenshot shows these items; the second shows the result after they were waived.

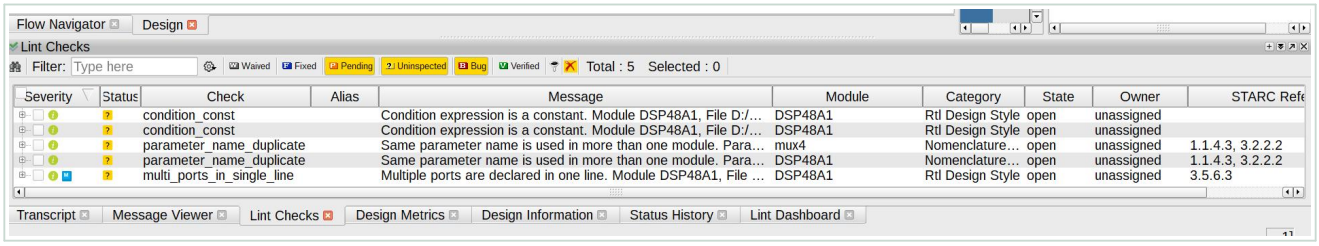



*Figure 9.1: Lint checks before waiving (5 info/style items)*

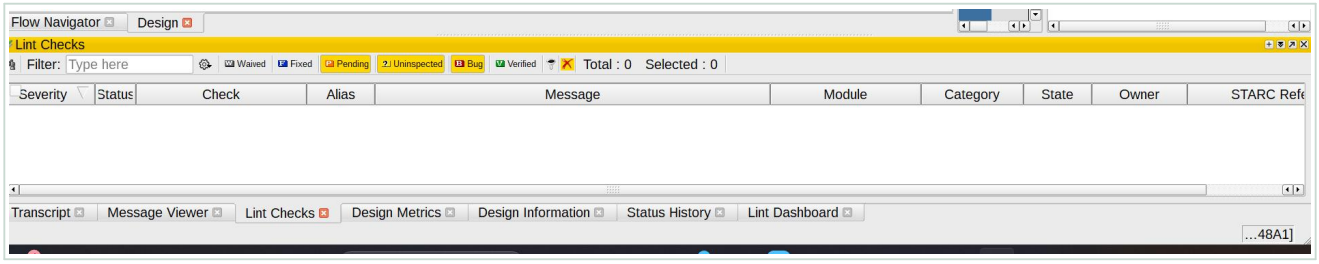



*Figure 9.2: Lint checks after waiving (0 remaining)*